

PROJECT UNDERSTANDING DOCUMENT

VeriCheck

Automated Fact-Checker for Vernacular News

Scalable AI Pipeline for Indian Language Misinformation Detection

Field	Detail
Document Type	Project Understanding & Complete Reference
Version	1.0 — March 2026
Project Name	VeriCheck
Domain	AI / NLP / Misinformation Detection
Target Languages	Hindi, Marathi, Tamil, Bengali, Telugu, Kannada, Gujarati, Punjabi
Cost Model	Mostly free/open-source — LLM API only for 5-10% of items

1. The Problem We Are Solving

Misinformation spreads rapidly on social media across India, particularly in vernacular languages. A false claim posted on WhatsApp or Twitter in Hindi or Tamil can reach millions of people within hours. Human fact-checkers cannot possibly keep up with the volume of content — there are simply too many posts and too few reviewers.

Existing AI fact-checking solutions face two critical problems:

- They are too slow — LLM-based verification takes seconds per item, which does not scale to millions of posts per day.
- They are too expensive — running every post through an LLM costs too much at scale.
- They get confused — retrieval systems return outdated or conflicting facts, leading to wrong verdicts.

1.1 The Core Challenge

Key Constraint 1 — Throughput

The system must process at least 5,000 posts per minute at peak load. A single LLM call takes 2-10 seconds. You cannot call the LLM for every post.

Key Constraint 2 — Context Accuracy

When retrieving verified facts, the system must not get confused by outdated or conflicting data in the facts database. A 2019 fact about a vaccine may directly contradict a 2024 fact about the same topic.

1.2 The Core Insight — Tiered Processing

The solution is a tiered pipeline. Instead of treating every post the same, we use cheap fast filters to eliminate most volume before reaching expensive LLM calls. About 60% of posts contain no verifiable factual claim at all — they are opinions, jokes, or questions. These are dropped cheaply at Tier 1. Of the remaining 40%, most can be auto-verified by matching against the facts database at Tier 2. Only 5-10% of total posts ever reach the expensive LLM tier. This keeps costs manageable while maintaining high accuracy.

2. System Architecture — VeriCheck Pipeline

VeriCheck is a five-tier processing pipeline. Every post enters at Tier 0 and flows downward. Each tier is progressively more expensive but handles progressively less volume. The architecture is the primary mechanism for achieving both the throughput constraint and the cost constraint simultaneously.

2.1 The Five Tiers

Tier	Name	Components	Volume Handled	Latency
0	Ingestion	Kafka, web scrapers, MCP connectors	100% of all posts	< 1ms
1	Pre-filter	fastText, spaCy NER, MinHash dedup	100% in, ~40% pass	< 5ms per item
2	Hybrid Retrieval	BM25 + FAISS + Cross-encoder reranker	~40% of posts	< 200ms per item
3	LLM Reasoning	Claude / GPT-4o batch API	5-10% of posts only	< 10 seconds
Output	Delivery	Dashboard, webhooks, human review queue	All verdicted items	< 10 seconds

2.2 What Happens at Each Tier

Tier 0 — Ingestion

Posts arrive from social media platforms and news sites via web scrapers and MCP connectors. Every post is immediately published to a Kafka topic called raw-posts. Kafka acts as a durable buffer — if any downstream service crashes, no posts are lost, and processing resumes from where it stopped. This is the entry point for 100% of all content.

Tier 1 — Pre-filter (the volume killer)

Three fast operations run on every post in under 5ms total:

1. Language detection using fastText — identifies which of the 8 supported Indian languages the post is written in. Unsupported languages are archived without further processing.
2. Claim extraction using spaCy NER with custom rules — identifies whether the post contains a verifiable factual claim. Opinions ('I think the PM is wrong'), jokes, and questions are dropped here. This eliminates approximately 60% of total volume.
3. Duplicate detection using MinHash — suppresses re-processing of posts that are near-identical to posts already in the pipeline. Prevents redundant work when the same claim goes viral.

Tier 2 — Hybrid Retrieval and Ranking

For posts that contain a real factual claim, the system searches the verified facts database using two methods simultaneously:

- BM25 keyword retrieval — finds facts that share specific keywords with the claim. Fast and precise for exact matches.
- FAISS dense vector retrieval — finds facts that are semantically similar even if they use different words. Handles paraphrasing and synonyms.

The top results from both methods are merged and then re-scored by a cross-encoder reranker. The reranker is the key to solving the context accuracy problem — it jointly scores each (claim, candidate fact) pair and applies penalties to facts that are outdated or from lower-authority sources. The reranker produces a confidence score. If the score is above a configured threshold, the post is auto-flagged and a verdict is generated without needing the LLM.

Tier 3 — LLM Reasoning (only 5-10% of posts)

Posts where the reranker confidence is below the threshold — meaning the system is genuinely uncertain — are sent to the LLM tier. The LLM receives a structured prompt containing the original claim, the top 3 retrieved facts from the database, and the language context. It returns a structured JSON verdict with a label, confidence score, evidence IDs, and a plain-language explanation. LLM calls use the batch API to minimize cost.

Output Layer

Verdicts flow to three destinations based on confidence level. High-confidence verdicts appear on the reviewer dashboard within 10 seconds. Low-confidence verdicts enter a human review queue. All verdicts are also available via a REST API and webhooks for third-party integrations. Human corrections from the review queue feed back into the verified facts database, continuously improving the system.

3. What is Kafka and Why We Use It

Apache Kafka is a distributed event streaming platform — essentially a very fast, durable message queue that sits between systems that produce data and systems that consume it. It was originally built by LinkedIn and is now open-source through the Apache Foundation.

3.1 The Mental Model

Think of Kafka as a conveyor belt at a factory. Multiple machines (your scrapers) drop items onto the belt. Multiple workstations (your pipeline workers) pick items off the belt to process them. Each workstation works at its own speed. Items are not removed from the belt until every workstation that needs them has processed them. And crucially — if one workstation breaks down, the belt keeps running, and that station can resume from exactly where it stopped once it is fixed.

3.2 Key Kafka Concepts

Concept	What it means	In VeriCheck
Topic	A named log of messages	raw-posts topic receives all scraped content
Partition	A parallel processing unit within a topic	10 partitions = 10 workers can process simultaneously
Producer	A service that writes messages to a topic	Your social media scrapers and MCP connectors
Consumer	A service that reads messages from a topic	Tier 1 pre-filter workers, analytics logger
Consumer Group	Multiple consumers sharing work on a topic	All Tier 1 workers form one group
Offset	Position in the log — where a consumer is up to	Lets crashed workers resume exactly where they stopped

3.3 Why Kafka Solves Your Throughput Problem

- Horizontal scaling: add more consumer workers to handle more volume instantly, with no code changes.
- Fault tolerance: if the LLM service goes down, Kafka holds all messages. Nothing is lost. Workers resume when the service recovers.
- Multiple independent consumers: your Tier 1 filter and your analytics logger both read the same topic at their own pace without interfering with each other.
- Replay: you can reprocess old posts with a new model version by rewinding the consumer offset.

Simpler alternative to start with

During development and early testing, use Redis Streams instead of Kafka. It implements the same concept (producer/consumer log), is much easier to set up locally, and has zero infrastructure overhead. Migrate to Kafka only when you need production-scale throughput.

4. Web MCPs in VeriCheck

MCP stands for Model Context Protocol — an open standard published by Anthropic that allows AI agents to call external tools (web scrapers, browsers, databases, APIs) as easily as calling a Python function. A Web MCP server wraps web browsing and scraping capability into a tool your pipeline can invoke directly.

4.1 Where MCPs Fit in the Pipeline

Place 1 — Ingestion Layer (replacing custom scrapers)

Traditional web scrapers are brittle — they break whenever a news site changes its HTML structure, which happens constantly. Web MCPs solve this problem. Instead of maintaining CSS selectors and XPath expressions for every site, you describe what you want in natural language and the MCP handles the rest, including JavaScript rendering, CAPTCHA bypass (for paid MCPs), and clean content extraction.

Place 2 — Live Fact Sourcing (keeping the database current)

When Tier 2 retrieval finds no high-confidence matching fact in the database, instead of immediately returning UNVERIFIABLE, the system can trigger a Web MCP to search trusted sources like PIB (Press Information Bureau), WHO, AltNews, or FactCheck.in in real time. The retrieved content is parsed, embedded, and stored in the facts database. This means the database grows automatically as new claims appear, without requiring manual curation.

4.2 Complete Free MCP Stack

MCP Server	Pipeline Stage	What It Does	Cost	Install Command
pskill9/web-search	Ingestion + Fact sourcing	Google search results, no API key required	Free	<code>npx mcp-web-search</code>
DuckDuckGo MCP	Ingestion + Fact sourcing	Unlimited web search, zero API key needed	Free	<code>npx mcp-duckduckgo</code>
SearXNG MCP	Ingestion + Fact sourcing	Self-hosted meta-search across multiple engines	Free	Docker self-host
Playwright MCP	Ingestion scraping	Full browser automation, handles JS-heavy portals	Free	<code>npx @playwright/mcp</code>
Puppeteer MCP	Ingestion scraping	Browser automation and screenshots	Free	<code>npx puppeteer-mcp</code>

MCP Server	Pipeline Stage	What It Does	Cost	Install Command
fetch MCP (official)	Fact sourcing	Fetch any URL and convert to clean markdown	Free	<code>npx @modelcontextprotocol/server-fetch</code>
PostgreSQL MCP	Storage	Query your verified facts database directly	Free	<code>npx @postgresql/mcp</code>
filesystem MCP	Storage	Read and write local files and logs	Free	<code>npx @modelcontextprotocol/server-filesystem</code>
crawl4ai MCP	Ingestion	LLM-ready crawling, returns structured JSON	Free	Docker self-host
Memory MCP	State	Store claim history across pipeline runs	Free	<code>npx @modelcontextprotocol/server-memory</code>

Recommended starter MCP setup (3 installs, zero cost)

1. `npx @modelcontextprotocol/server-fetch` — fetch any URL as clean text 2. `npx mcp-duckduckgo` — web search with no API key 3. `npx @playwright/mcp` — full browser for JS-heavy sites These three cover ingestion, fact sourcing, and scraping with zero cost and no API keys required.

5. Google Cloud — Not Needed

Google has released official MCP servers for its enterprise services including Google Workspace, BigQuery, Cloud SQL, and Google Maps. These are useful for organizations already paying for Google Cloud. However, for VeriCheck, you do not need any of them.

5.1 Why Google's APIs Do Not Work for VeriCheck

Google Service	The Problem	Why It Fails for VeriCheck
Google Custom Search API	Only 100 free queries per day	VeriCheck needs thousands of searches per minute — 100/day is useless
Custom Search JSON API	Closed to new customers	You cannot even sign up — it is being discontinued
Google Cloud MCPs	Enterprise subscription required	Requires existing Google Cloud billing to be enabled
Google Maps MCP	Paid API key required	Not relevant to fact-checking anyway

5.2 Your Complete Free Stack — No Google, No Subscription, No Credit Card

Pipeline Need	Free Solution	How to Install
Web search	DuckDuckGo MCP — no key, unlimited queries	<code>npx mcp-duckduckgo</code>
URL content fetch	fetch MCP by Anthropic — official, zero cost	<code>npx @modelcontextprotocol/server-fetch</code>
JS-heavy news sites	Playwright MCP — open source browser automation	<code>npx @playwright/mcp</code>
Meta-search engine	SearXNG — self-hosted, searches Bing + DDG + more	Docker: <code>docker run searxng/searxng</code>
Database queries	PostgreSQL MCP — connects to your own DB	<code>npx @postgresql/mcp</code>
File operations	filesystem MCP by Anthropic — official, free	<code>npx @modelcontextprotocol/server-filesystem</code>

6. Complete Technology Stack

Every tool in VeriCheck is free and open-source, with the single exception of the LLM API which is only called for 5-10% of posts and can be further minimized using batch pricing.

6.1 Core Pipeline Components

Component	Tool / Library	Purpose	Cost
Message queue	Apache Kafka	Durable queue between ingestion and processing workers	Free
Queue alternative (dev)	Redis Streams	Simpler Kafka alternative for development and testing	Free
Language detection	fastText by Meta	Detects 170+ languages in under 1ms per post	Free
Claim extraction	spaCy + custom NER	Identifies factual claims from vernacular text	Free
Duplicate detection	datasketch MinHash	Suppresses near-duplicate posts probabilistically	Free
Keyword retrieval	rank_bm25 (BM25)	Fast exact keyword matching against facts database	Free
Vector search	FAISS by Meta	Semantic similarity search across millions of fact embeddings	Free
Embeddings model	sentence-transformers	Multilingual text embeddings covering Indian languages	Free
Reranker	MiniLM cross-encoder	Re-scores (claim, fact) pairs jointly for high accuracy	Free
Primary database	PostgreSQL + pgvector	Stores facts with ACID compliance and vector search	Free
Cache	Redis	Sub-millisecond hot fact lookup and bloom filter storage	Free
LLM reasoning	Claude or GPT-4o API	Deep reasoning on ambiguous claims — 5-10% of items only	Paid (small)
API framework	FastAPI (Python)	Async REST API with automatic OpenAPI documentation	Free
Web scraping MCPs	Playwright, fetch, DuckDuckGo MCPs	Scraping and search without brittle CSS selectors	Free
Monitoring	Prometheus + Grafana	Real-time metrics on throughput, latency, accuracy	Free
Orchestration	Docker + Kubernetes	Container management and horizontal autoscaling	Free

7. Where to Get the Code

All components of VeriCheck are available as open-source repositories on GitHub or via npm/pip package managers. Below is the complete list ordered by when you should integrate each component in your build.

7.1 GitHub Repositories — Ordered by Build Priority

Priority	Repository	GitHub URL	What You Get
1st	Official MCP Servers (Anthropic)	github.com/modelcontextprotocol/servers	fetch, filesystem, memory, sequential-thinking MCPs — start here
2nd	sentence-transformers	github.com/UKPLab/sentence-transformers	Multilingual embeddings + cross-encoder reranker models
3rd	FAISS	github.com/facebookresearch/faiss	Vector similarity search engine for your embeddings index
4th	Playwright MCP (Microsoft)	github.com/microsoft/playwright-mcp	Full browser automation MCP for scraping JS-heavy portals
5th	fastText	github.com/facebookresearch/fastText	Language detection in under 1ms, supports all Indian languages
6th	crawl4ai	github.com/unclecode/crawl4ai	LLM-ready web crawling, returns structured JSON, MCP-compatible
7th	spaCy models (Indian)	github.com/explosion/spacy-models	Pre-trained NER models, fine-tune for claim extraction
8th	pgvector	github.com/pgvector/pgvector	PostgreSQL extension for storing and querying vector embeddings
9th	datasketch	github.com/ekzhu/datasketch	MinHash implementation for duplicate detection
10th	rank_bm25	github.com/dorianbrown/rank_bm25	BM25 keyword retrieval for the hybrid search layer

7.2 Install Commands — Quick Reference

Component	Install Command
sentence-transformers (embeddings + reranker)	pip install sentence-transformers
FAISS (CPU version)	pip install faiss-cpu
spaCy + Hindi model	pip install spacy && python -m spacy download xx_ent_wiki_sm
fastText	pip install fasttext
pgvector Python client	pip install pgvector
datasketch (MinHash)	pip install datasketch
rank_bm25	pip install rank_bm25
FastAPI + Uvicorn	pip install fastapi uvicorn
Kafka Python client	pip install kafka-python
fetch MCP (official)	npx @modelcontextprotocol/server-fetch
Playwright MCP	npx @playwright/mcp
DuckDuckGo MCP	npx mcp-duckduckgo
filesystem MCP	npx @modelcontextprotocol/server-filesystem

7.3 Hugging Face — Pre-trained Models

These models can be downloaded directly from Hugging Face (huggingface.co/models) and loaded via the sentence-transformers library:

- `paraphrase-multilingual-MiniLM-L12-v2` — multilingual sentence embeddings, covers all 8 target languages
- `cross-encoder/ms-marco-MiniLM-L-6-v2` — the reranker model for Tier 2
- `ai4bharat/indic-bert` — BERT model fine-tuned specifically on Indian language text
- `google/muril-base-cased` — multilingual representations for Indian languages

8. Recommended Build Order

Do not try to build everything at once. Follow this sequence — each step produces something testable before you add complexity.

Phase	What to Build	What You Can Test	Time Estimate
Phase 1	PostgreSQL + pgvector + seed 1,000 verified facts manually	Can you store and retrieve facts by vector similarity?	1-2 days
Phase 2	sentence-transformers embeddings + BM25 + FAISS index	Does retrieval return relevant facts for a test claim?	2-3 days
Phase 3	Cross-encoder reranker + confidence gate	Does reranking improve result quality? Is the threshold sensible?	2 days
Phase 4	FastAPI verdict endpoint + basic dashboard	End-to-end: submit claim, get verdict back via API	2-3 days
Phase 5	fastText language detection + spaCy claim extractor	Does Tier 1 correctly drop opinions and keep factual claims?	3-4 days
Phase 6	Playwright MCP + fetch MCP for scraping	Can you scrape a news site and extract clean text automatically?	1-2 days
Phase 7	Redis Streams queue (Kafka later)	Can multiple workers process posts in parallel from a queue?	2 days
Phase 8	LLM tier integration (Claude batch API)	Does the LLM produce correct structured verdicts for hard cases?	2-3 days
Phase 9	Kafka replace Redis Streams + Kubernetes deploy	Does the system hold up under 5,000 posts per minute load test?	1 week

The golden rule

Build Phase 1 through 4 first — this gives you a working fact-checker that can verify claims against a static database. Everything from Phase 5 onward adds scale, automation, and robustness. A working Phase 1-4 system is already more valuable than a half-built Phase 1-9 system.

9. Verdict System

Every post that passes through the pipeline receives one of four labels plus a confidence score and evidence chain.

Label	Meaning	Example	Action
TRUE	Claim matches verified facts with high confidence	Post says PM inaugurated X bridge on date Y — PIB confirms	Pass — no action
FALSE	Claim directly contradicts verified facts	Post says vaccine causes infertility — WHO, ICMR deny this	Flag immediately — dashboard alert
MISLEADING	Claim is technically true but omits critical context	Post shows real statistic but cherry-picked time period	Flag with explanation
UNVERIFIABLE	No matching verified fact found and LLM cannot determine	Claim about local politician with no authoritative source	Route to human reviewer

9.1 Verdict JSON Structure

Every verdict produced by the system follows this structure:

```
{ "verdict_id": "uuid", "item_id": "uuid", "label": "FALSE", "confidence": 0.94,
  "evidence": [{ "fact_id": "uuid", "text": "...", "source": "who.int", "date":
    "2024-01-15" }], "explanation": "The claim contradicts WHO guidance published
  January 2024.", "tier_reached": 2, "model_version": "v1.2", "human_reviewed": false
}
```

10. Key Design Decisions Explained

Decision	What we chose	Why
Why hybrid retrieval?	BM25 + FAISS together	BM25 is precise for keywords; FAISS handles paraphrasing. Together they have higher recall than either alone.
Why a cross-encoder reranker?	MiniLM-L-6 cross-encoder	Bi-encoders (used in FAISS) score query and document independently. Cross-encoders see both together — much higher accuracy for short fact-check pairs.
Why not call LLM for everything?	Tier 3 limited to 5-10%	At 5,000 posts/min, calling LLM for all would cost thousands of dollars per day. The confidence gate filters to only the genuinely ambiguous cases.
Why Kafka over a simple queue?	Kafka for production	Kafka persists messages, supports replay, and scales to millions of messages. Simple queues lose messages on crash. Use Redis Streams in dev.
Why free MCPs over paid?	DuckDuckGo + Playwright + fetch	Google's Custom Search is closed to new customers. Firecrawl requires a paid key. The free stack handles all scraping needs with no rate limits.
Why PostgreSQL + pgvector?	Not a dedicated vector DB	pgvector gives you vector search inside PostgreSQL — ACID compliance, SQL joins, versioned facts, and vector similarity in one database. Simpler than running Pinecone or Weaviate separately.

